Q1. Longest Substring Without Repeating Characters

Domain: Python / DSA **Difficulty:** Medium

Problem Statement

Given a string s, return the length of the longest substring that contains no repeated characters.

Input Format

A single string s.

Output Format

Print one integer: the maximum length of a substring with all unique characters.

Constraints

0 <= len(s) <= 5 × 10^4. The string may contain letters, digits, symbols, and spaces.

Expected Complexity

Expected O(n) time.

Public Test Cases

Public Test Case 1

Input

```
abcabcbb
```

Expected Output

```
3
```

Public Test Case 2

Input

```
bbbbb
```

Expected Output

```
1
```

Q2. Group Anagrams

Domain: Python / Hashing **Difficulty:** Medium

Problem Statement

Given an array of strings, group the strings that are anagrams of each other. Group and element ordering are not significant.

Input Format

First line: n. Next n lines: one lowercase string per line.

Output Format

Print the groups of anagrams, with strings separated by spaces.

Constraints

1 <= n <= 10^4. Each string has length <= 100 and contains lowercase English letters.

Expected Complexity

Expected O(n × k) time, where k is average string length.

Public Test Cases

Public Test Case 1

Input

```
6
eat
tea
tan
ate
nat
bat
```

Expected Output

```
eat tea ate
tan nat
bat
```

Public Test Case 2

Input

```
4
listen
silent
enlist
google
```

Expected Output

```
listen silent enlist
google
```

Q3. Implement an LRU Cache

Domain: Python / Data Structures **Difficulty:** Hard

Problem Statement

Design a Least Recently Used cache supporting GET and PUT. When capacity is exceeded, evict the least recently used key.

Input Format

First line: capacity and number of operations. Each following line is GET key or PUT key value.

Output Format

For every GET, print the value or -1 if the key does not exist.

Constraints

1 <= capacity <= 10^5. Up to 2 × 10^5 operations.

Expected Complexity

Required O(1) average time for GET and PUT.

Public Test Cases

Public Test Case 1

Input

```
2 6
PUT 1 10
PUT 2 20
GET 1
PUT 3 30
GET 2
GET 3
```

Expected Output

```
10
-1
30
```

Public Test Case 2

Input

```
1 5
PUT 1 100
GET 1
PUT 2 200
GET 1
GET 2
```

Expected Output

```
100
-1
200
```

Q4. Analyze Server Requests

Domain: Python / Hashing **Difficulty:** Hard

Problem Statement

Given server logs containing an IP address and HTTP status code, find the most active IP, total 4xx responses, and total 5xx responses. On a tie, choose the lexicographically smallest IP.

Input Format

First line: n. Next n lines: IP_ADDRESS STATUS_CODE.

Output Format

Print the most active IP, the number of 4xx responses, and the number of 5xx responses, one per line.

Constraints

1 <= n <= 10^6.

Expected Complexity

Expected O(n) time and O(k) space.

Public Test Cases

Public Test Case 1

Input

```
6
192.168.1.10 200
192.168.1.11 404
192.168.1.10 500
192.168.1.12 404
192.168.1.11 200
192.168.1.10 200
```

Expected Output

```
192.168.1.10
2
1
```

Public Test Case 2

Input

```
5
10.0.0.2 404
10.0.0.1 404
10.0.0.2 200
10.0.0.1 200
10.0.0.3 503
```

Expected Output

```
10.0.0.1
2
1
```

Q5. Top K Frequent Elements

Domain: DSA / Hashing / Heap **Difficulty:** Medium

Problem Statement

Given an integer array nums and k, return the k most frequent elements. If frequencies tie, any order is accepted.

Input Format

First line: n and k. Second line: n integers.

Output Format

Print the k selected elements separated by spaces.

Constraints

1 <= n <= 10^5. 1 <= k <= number of distinct elements.

Expected Complexity

Expected O(n log k) or O(n) time.

Public Test Cases

Public Test Case 1

Input

```
6 2
1 1 1 2 2 3
```

Expected Output

```
1 2
```

Public Test Case 2

Input

```
9 3
4 4 1 1 1 2 2 2 2
```

Expected Output

```
2 1 4
```

Q6. Detect a Cycle in a Directed Graph

Domain: DSA / Graphs **Difficulty:** Hard

Problem Statement

Given a directed graph with V vertices and E edges, determine whether the graph contains a cycle.

Input Format

First line: V E. Next E lines: u v representing u -> v.

Output Format

Print YES if the graph contains a directed cycle; otherwise print NO.

Constraints

1 <= V <= 10^5. 0 <= E <= 2 × 10^5.

Expected Complexity

Expected O(V + E) time.

Public Test Cases

Public Test Case 1

Input

```
3 3
0 1
1 2
2 0
```

Expected Output

```
YES
```

Public Test Case 2

Input

```
4 3
0 1
1 2
2 3
```

Expected Output

```
NO
```

Q7. Minimum Cost Path in a Grid

Domain: DSA / Dynamic Programming **Difficulty:** Hard

Problem Statement

Given an m × n grid of positive costs, move only right or down from the top-left to the bottom-right. Return the minimum path cost including both endpoints.

Input Format

First line: m n. Next m lines contain n positive integers.

Output Format

Print the minimum possible path cost.

Constraints

1 <= m,n <= 500. Each cell cost is between 1 and 10^4.

Expected Complexity

Expected O(m × n) time; O(n) auxiliary space is possible.

Public Test Cases

Public Test Case 1

Input

```
3 3
1 3 1
1 5 1
4 2 1
```

Expected Output

```
7
```

Public Test Case 2

Input

```
2 3
1 2 3
4 5 6
```

Expected Output

```
12
```

Q8. Network Shortest Path

Domain: DSA / Graphs / Greedy **Difficulty:** Hard

Problem Statement

Given a directed weighted graph with non-negative latencies and a source S, compute the shortest distance from S to every vertex.

Input Format

First line: V E S. Next E lines: U V W.

Output Format

Print V distances in vertex order. Print -1 for unreachable vertices.

Constraints

1 <= V <= 10^5. 0 <= E <= 2 × 10^5. 0 <= W <= 10^6.

Expected Complexity

Expected O((V + E) log V) using Dijkstra.

Public Test Cases

Public Test Case 1

Input

```
4 5 0
0 1 4
0 2 2
2 1 1
1 3 2
2 3 5
```

Expected Output

```
0 3 2 5
```

Public Test Case 2

Input

```
5 4 2
2 0 7
2 1 2
1 3 3
3 4 1
```

Expected Output

```
7 2 0 5 6
```

Q9. Design a Banking Account System

Domain: OOP **Difficulty:** Medium

Problem Statement

Design an object-oriented banking system with a common BankAccount base class and SavingsAccount and CurrentAccount subclasses. Support deposit, withdraw, and balance retrieval. Savings accounts enforce a minimum balance; current accounts may allow an overdraft limit.

Input Format

Provide account type, account id, initial balance, account-specific limit, and a sequence of operations as specified by your implementation.

Output Format

For each transaction print its status, then print the final balance.

Constraints

Use encapsulation, inheritance, constructors, overriding, and polymorphism. Do not directly manipulate internal balance fields.

Expected Complexity

Individual account operations should be O(1).

Public Test Cases

Public Test Case 1

Input

```
SavingsAccount S101 5000 1000
WITHDRAW 3500
BALANCE
```

Expected Output

```
SUCCESS
1500
```

Public Test Case 2

Input

```
CurrentAccount C201 1000 500
WITHDRAW 1300
BALANCE
```

Expected Output

```
SUCCESS
-300
```

Q10. Employee Payroll System

Domain: OOP / Polymorphism **Difficulty:** Hard

Problem Statement

Design a payroll system using an abstract Employee base class and specialized employee types. Each type calculates a role-specific bonus and final salary equals base salary plus bonus.

Input Format

First line: n. Next n lines: employee type, employee id, name, base salary. Types: Developer, Manager, Intern.

Output Format

For each employee print id, bonus, final salary, then print total payroll.

Constraints

Developer bonus = 10%, Manager = 20%, Intern = 5%. Demonstrate abstraction, inheritance, encapsulation, overriding, and runtime polymorphism. Avoid a large type-checking chain.

Expected Complexity

Expected O(n) time.

Public Test Cases

Public Test Case 1

Input

```
3
Developer 101 Rahul 60000
Manager 102 Priya 90000
Intern 103 Arjun 20000
```

Expected Output

```
101 6000 66000
102 18000 108000
103 1000 21000
TOTAL 195000
```

Public Test Case 2

Input

```
2
Manager 201 Anika 50000
Developer 202 Ravi 40000
```

Expected Output

```
201 10000 60000
202 4000 44000
TOTAL 104000
```